Fundamentals of PYTHON DATA STRUCTURES

2ND EDITION

Kenneth A. Lambert

National Economics University
Faculty of Mathematical Economics

Fundamentals of Python: Data Structures

- Instructor: Duc Minh Vu (MFE)
- Email: minhvd [at] neu.edu.vn

Learning Objectives

After completing this chapter, you will be able to:

1. Measure the performance of an algorithm by obtaining running times and instruction counts with different data sets
2. Analyze an algorithm's performance by determining its order of complexity, using big-O notation
3. Distinguish the common orders of complexity and the algorithmic patterns that exhibit them

Learning Objectives

After completing this chapter, you will be able to:

1. Distinguish between the improvements obtained by tweaking an algorithm and reducing its order of complexity
2. Write a simple linear search algorithm and a simple sort algorithm

11.1 Measuring the Efficiency of Algorithms

- Measuring the Run Time of an Algorithm
  - use the computer's clock to obtain an actual run time
  - benchmarking or profiling

```
"""
File: timing1.py
Prints the running times for problem sizes that double,
using a single loop.
"""

import time

problemSize = 10000000
print "%12s16s" % ("Problem Size", "Seconds")
for count in xrange(5):

    start = time.time()
    # The start of the algorithm
    work = 1
    for x in xrange(problemSize):
        work += 1
        work -= 1
    # The end of the algorithm
    elapsed = time.time() - start

    print "%12d%16.3f" % (problemSize, elapsed)
    problemSize *= 2
```

```
Problem Size      Seconds
    10000000          3.8
    20000000        7.591
    40000000       15.352
    80000000       30.697
   160000000       61.631
```

[FIGURE 11.1] The output of the tester program

```
for j in xrange(problemSize):
    for k in xrange(problemSize):
        work += 1
        work -= 1
```

```
Problem Size      Seconds
        1000        0.387
        2000        1.581
        4000        6.463
        8000       25.702
       16000      102.666
```

[FIGURE 11.2] The output of the second tester program with a nested loop and initial problem size of 1000

Note that when the problem size doubles, the number of seconds of running time more or less quadruples. At this rate, it would take 175 days to process the largest number in the previous data set!

11.1.2 Counting Instructions

- Count the instructions executed with different problem sizes.
- Count the instructions in the high-level code in which the algorithm is written, not instructions in the executable machine language program.
- Distinguish between two classes of instructions:
  - Instructions that execute the same number of times regardless of the problem size
  - Instructions whose execution count varies with the problem size

```
"""
File: counting.py
Prints the number of iterations for problem sizes
that double, using a nested loop.
"""

problemSize = 1000
print "%12s%15s" % ("Problem Size", "Iterations")
for count in xrange(5):
    number = 0

    # The start of the algorithm
    work = 1
    for j in xrange(problemSize):
        for k in xrange(problemSize):
            number += 1
            work += 1
            work -= 1
   # The end of the algorithm

    print "%12d%15d" % (problemSize, number)
    problemSize *= 2
```

```
Problem Size     Iterations
        1000        1000000
        2000        4000000
        4000       16000000
        8000       64000000
       16000      256000000
```

[FIGURE 11.3] The output of a tester program that counts iterations

```
"""
File: countfib.py
Prints the number of calls of a recursive Fibonacci
function with problem sizes that double.
"""

class Counter(object):
    """Tracks a count."""

    def __init__(self):
        self._number = 0

    def increment(self):
        self._number += 1

    def __str__(self):
        return str(self._number)

def fib(n, counter):
    """Count the number of calls of the Fibonacci
    function."""
    counter.increment()
    if n < 3:
        return 1
    else:
        return fib(n - 1, counter) + fib(n - 2, counter)

problemSize = 2
print "%12s%15s" % ("Problem Size", "Calls")
for count in xrange(5):
    counter = Counter()

    # The start of the algorithm
    fib(problemSize, counter)
    # The end of the algorithm

    print "%12d%15s" % (problemSize, counter)
    problemSize *= 2
```

```
Problem Size        Calls
           2            1
           4            5
           8           41
          16         1973
          32      4356617
```

[FIGURE 11.4] The output of a tester program that runs the Fibonacci function

11.1 Exercises

1 Write a tester program that counts and displays the number of iterations of the following loop:

```
while problemSize > 0:
    problemSize = problemSize / 2
```

2 Run the program you created in Exercise 1 using problem sizes of 1000, 2000, 4000, 10,000, and 100,000. As the problem size doubles or increases by a factor of 10, what happens to the number of iterations?

3 The difference between the results of two calls of the **`time`** function **`time()`** is an elapsed time. Because the operating system might use the CPU for part of this time, the elapsed time might not reflect the actual time that a Python code segment uses the CPU. Browse the Python documentation for an alternative way of recording the processing time and describe how this would be done.

11.2 Complexity Analysis

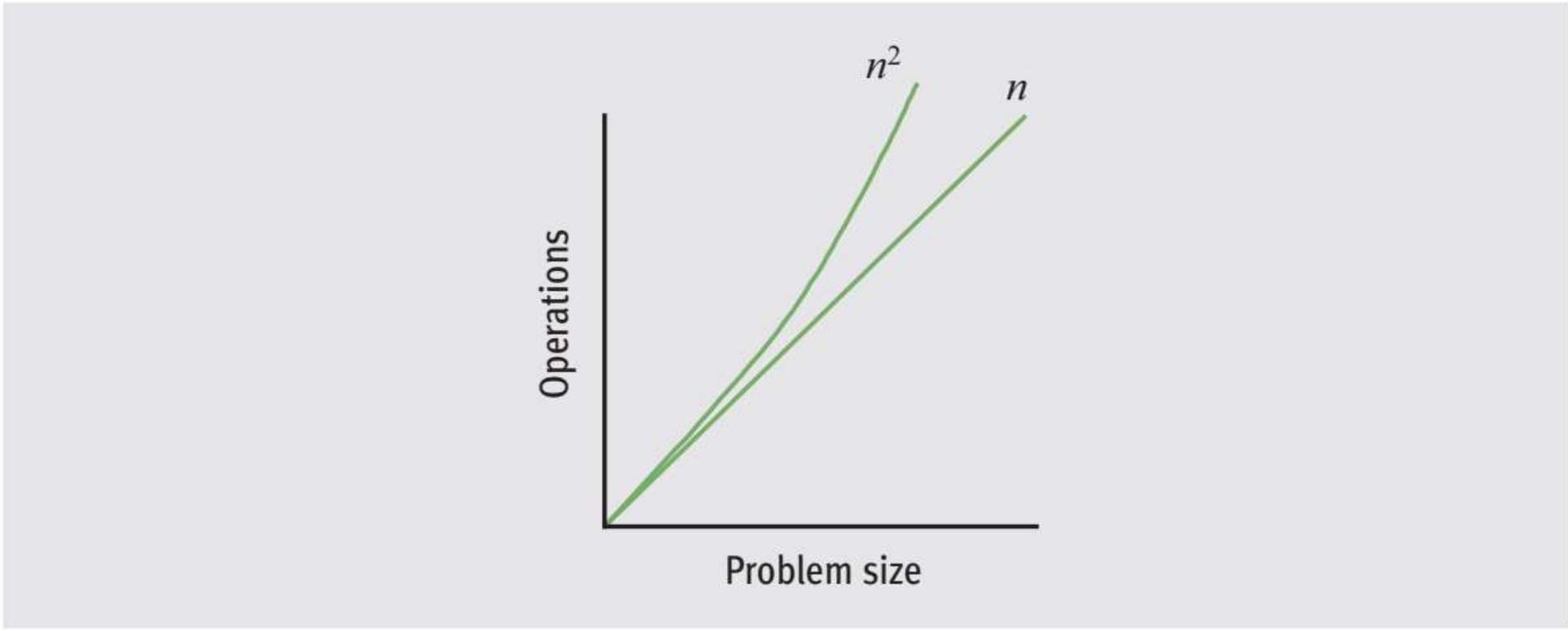
- A method of determining the efficiency of algorithms
- Allows to rate algorithms independently of platform-dependent timings or impractical instruction counts.

11.2.1 Orders of Complexity

- Level of increasing when the size of the input increases

PROBLEM SIZE	WORK OF THE FIRST ALGORITHM	WORK OF THE SECOND ALGORITHM
2	2	4
10	10	100
1000	1000	1,000,000

[TABLE 11.1] The amounts of work in the tester programs



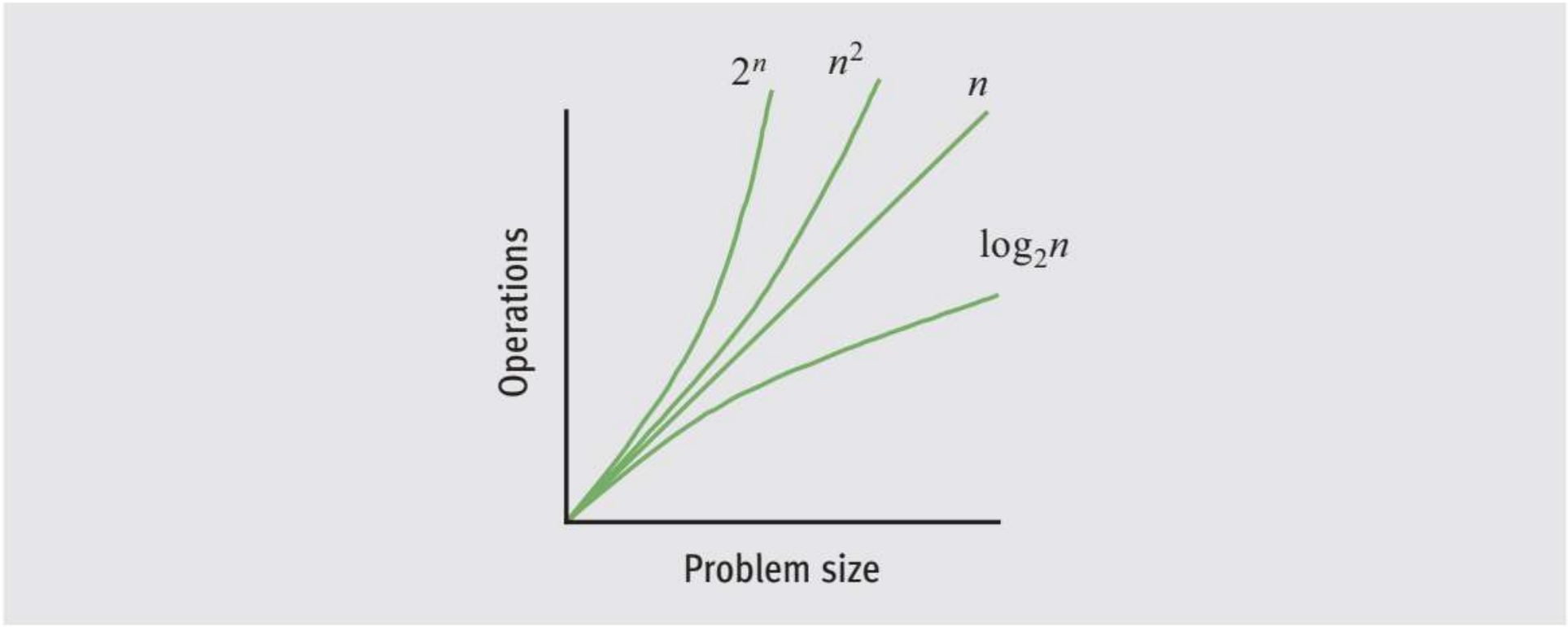
[FIGURE 11.5] A graph of the amounts of work done in the tester programs

11.2.1 Orders of Complexity

- Some popular order: const, linear, quadratic, logarithmic, polynomial, exponential

n	LOGARITHMIC ($\log_2 n$)	LINEAR (n)	QUADRATIC (n^2)	EXPONENTIAL (2^n)
100	7	100	10,000	Off the charts
1000	10	1000	1,000,000	Off the charts
1,000,000	20	1,000,000	1,000,000,000,000	Really off the charts

[TABLE 11.2] Some sample orders of complexity



[FIGURE 11.6] A graph of some sample orders of complexity

11.2.2 Big-O Notation

- Can not count or evaluate precisely the number of operations
- Focus on one term as dominant.
  - As input size becomes large, the dominant term becomes so large that the amount of work represented by the other terms can be ignore
- Asymptotic analysis: the value of the function asymptotically approaches or approximates the value of its largest term as the input size becomes very large.
- big-O notation: “O” stands for “on the order of,” a reference to the order of complexity of the work of the algorithm

11.2.3 The Role of the Constant of Proportionality

- Constant of proportionality involves the terms and coefficients that are usually ignored during big-O analysis.
- However, when these items are large, they may have an impact on the algorithm, particularly for small and medium-sized data sets
- Determine the constant?

```
work = 1
for x in xrange(problemSize):
    work += 1
    work -= 1
```

11.2 Exercises

1 Assume that each of the following expressions indicates the number of operations performed by an algorithm for a problem size of n. Point out the dominant term of each algorithm and use big-O notation to classify it.

a $2^n - 4n^2 + 5n$
b $3n^2 + 6$
c $n^3 + n^2 - n$

2 For problem size n, algorithms A and B perform n^2 and $\frac{1}{2}\, n^2 + \frac{1}{2}\, n$ instructions, respectively. Which algorithm does more work? Are there particular problem sizes for which one algorithm performs significantly better than the other? Are there particular problem sizes for which both algorithms perform approximately the same amount of work?

3 At what point does an n^4 algorithm begin to perform better than a 2^n algorithm?

11.3 Search Algorithms

- Search for a Minimum

```
def ourMin(lyst):
    """Returns the position of the minimum item."""
    minpos = 0
    current = 1
    while current < len(lyst):
        if lyst[current] < lyst[minpos]:
            minpos = current
        current += 1
    return minpos
```

11.3 Search Algorithms

11.3 Search Algorithms

- Ignore three instructions outside the loop that execute the same number of times regardless of the size of the list.
- The comparison in the **if** statement and the increment of **current** execute on each pass through the loop.
- Thus, the algorithm must make $n - 1$ comparisons for a list of size n.
- Therefore, the algorithm's complexity is $O(n)$

11.3 Search Algorithms

11.3.2 Linear Search of a List

```
def linearSearch(target, lyst):
    """Returns the position of the target item if found,
    or -1 otherwise."""
    position = 0
    while position < len(lyst):
        if target == lyst[position]:
            return position
        position += 1
    return -1
```

11.3 Search Algorithms

11.3.2 Linear Search of a List

```
def linearSearch(target, lyst):
    """Returns the position of the target item if found,
    or -1 otherwise."""
    position = 0
    while position < len(lyst):
        if target == lyst[position]:
            return position
        position += 1
    return -1
```

11.3.3 Best-Case, Worst-Case, and Average-Case Performance

- We focus more about average and worst-case performances than about best-case performances.

1 In the worst case, the target item is at the end of the list or not in the list at all. Then the algorithm must visit every item and perform n iterations for a list of size n. Thus, the worst-case complexity of a linear search is $O(n)$.

2 In the best case, the algorithm finds the target at the first position, after making one iteration, for an $O(1)$ complexity.

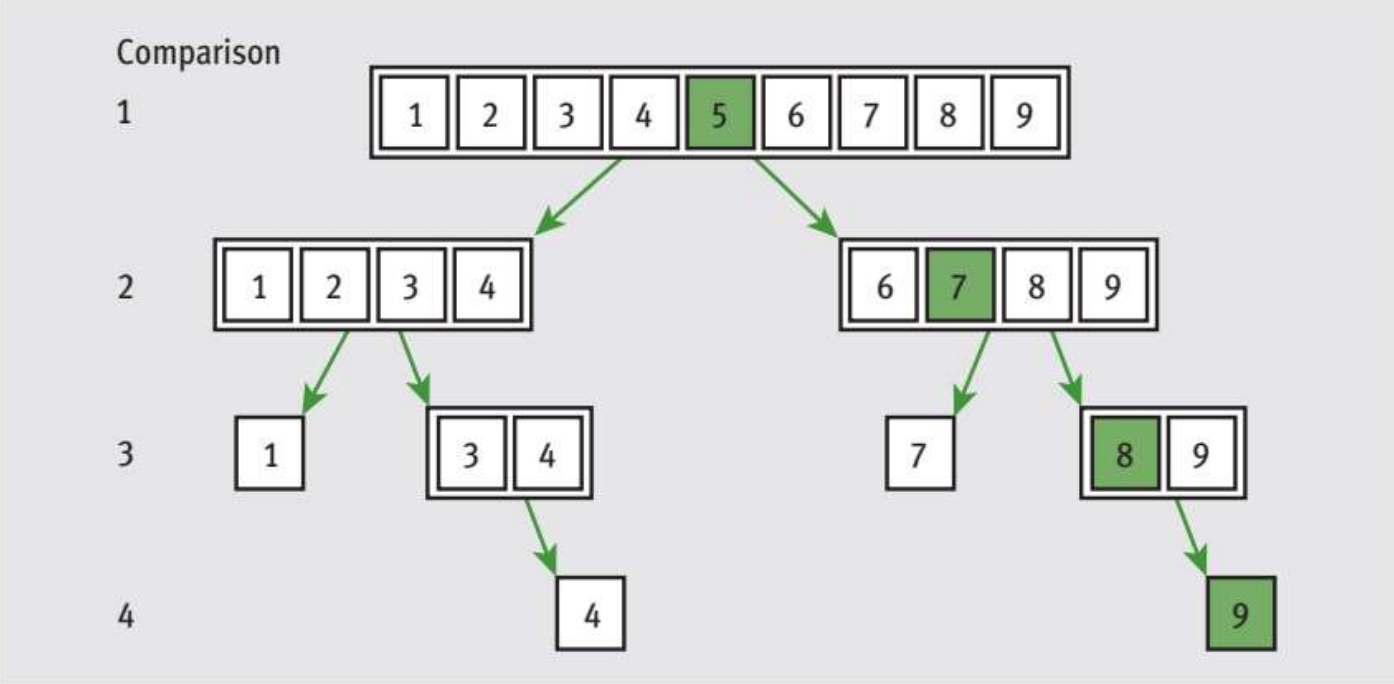
3 To determine the average case, you add the number of iterations required to find the target at each possible position and divide the sum by n. Thus, the algorithm performs $(n + n - 1 + n - 2 + \ldots + 1) / n$, or $(n + 1) / 2$ iterations. For very large n, the constant factor of /2 is insignificant, so the average complexity is still $O(n)$.

11.3.4 Binary Search of a List

- When searching sorted data, you can use a binary search.
- Assume that the items in the list are sorted in ascending order.
- The search algorithm goes to the middle position in the list and compares the item at that position to the target.
- If there is a match, the algorithm returns the position.
- Otherwise, if the target is less than the current item, the algorithm searches the portion of the list before the middle position.
- If the target is greater than the current item, the algorithm searches the portion of the list after the middle position.
- The search process stops when the target is found or the current beginning position is greater than the current ending position.

```
def binarySearch(target, lyst):
    left = 0
    right = len(lyst) - 1
    while left <= right:
        midpoint = (left + right) / 2
        if target == lyst[midpoint]:
            return midpoint
        elif target < lyst[midpoint]:
            right = midpoint - 1
        else:
            left = midpoint + 1
    return -1
```

There is just one loop with no nested or hidden loops. Once again, the worst case occurs when the target is not in the list. How many times does the loop run in the worst case? This is equal to the number of times the size of the list can be divided by 2 until the quotient is 1. For a list of size n, you essentially perform the reduction $n / 2 / 2 \ldots / 2$ until the result is 1. Let k be the number of times we divide n by 2. To solve for k, you have $n / 2^k = 1$, and $n = 2^k$, and $k = \log_2 n$. Thus, the worst-case complexity of binary search is $O(\log_2 n)$.



[FIGURE 11.7] The items of a list visited during a binary search for 10

11.3 Exercises

1 Suppose that a list contains the values

```
20  44  48  55  62  66  74  88  93  99
```

at index positions 0 through 9. Trace the values of the variables **`left`**, **`right`**, and **`midpoint`** in a binary search of this list for the target value 90. Repeat for the target value 44.

2 The method we usually use to look up an entry in a phone book is not exactly the same as a binary search because, when using a phone book, we don't always go to the midpoint of the sublist being searched. Instead, we estimate the position of the target based on the alphabetical position of the first letter of the person's last name. For example, when we are looking up a number for "Smith," we look toward the middle of the second half of the phone book first, instead of in the middle of the entire book. Suggest a modification of the binary search algorithm that emulates this strategy for a list of names. Is its computational complexity any better than that of the standard binary search?

11.4 Sort Algorithms

11.4 Sort Algorithms

- Computer scientists have devised many ingenious strategies for sorting a list of items.
- We examine some algorithms that are easy to write but are inefficient.
- Each of the Python sort functions that we develop here operates on a list of integers and uses a swap function to exchange the positions of two items in the list

```
def swap(lyst, i, j):
    """Exchanges the items at positions i and j."""
    # You could say lyst[i], lyst[j] = lyst[j], lyst[i]
    # but the following code shows what is really going on
    temp = lyst[i]
    lyst[i] = lyst[j]
    lyst[j] = temp
```

11.4.1 Selection Sort

- Each pass of the algorithm selects a single item to be moved.
- Pass *k* moves/swaps the k-th smallest element to its correct position.

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
5	1*	1	1	1
3	3	2*	2	2
1	5*	5	3*	3
2	2	3*	5*	4*
4	4	4	4	5*

[TABLE 11.3] A trace of the data during a selection sort

```
def selectionSort(lyst):
    i = 0
    while i < len(lyst) - 1:          # Do n - 1 searches
        minIndex = i                  # for the smallest
        j = i + 1
        while j < len(lyst):          # Start a search
            if lyst[j] < lyst[minIndex]:
                minIndex = j
            j += 1
        if minIndex != i:             # Exchange if needed
            swap(lyst, minIndex, i)
        i += 1
```

This function includes a nested loop. For a list of size n, the outer loop executes $n - 1$ times. On the first pass through the outer loop, the inner loop executes $n - 1$ times. On the second pass through the outer loop, the inner loop executes $n - 2$ times. On the last pass through the outer loop, the inner loop executes once. Thus, the total number of comparisons for a list of size n is the following:

$$(n - 1) + (n - 2) + \ldots + 1 =$$

$$n\ (\mathrm{n} - 1) / 2 =$$

$$\tfrac{1}{2}\ n^2 - \tfrac{1}{2}\ n$$

For large n, you can pick the term with the largest degree and drop the coefficient, so selection sort is $O(n^2)$ in all cases. For large data sets, the cost of swapping items might also be significant. Because data items are swapped only in the outer loop, this additional cost for selection sort is linear in the worst and average cases.

11.4.2 Bubble Sort

- Compare two consecutive items
- Each time the items in the pair are out of order, the algorithm swaps them
- The effect of bubbling the largest items to the end of the list.
- The k-th iteration will bubble the k-th largest element to its correct position

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
5	4*	4	4	4
4	5*	2*	2	2
2	2	5*	1*	1
1	1	1	5*	3*
3	3	3	3	5*

[TABLE 11.4] A trace of the data during a bubble sort

```
def bubbleSort(lyst):
    n = len(lyst)
    while n > 1:                        # Do n - 1 bubbles
        i = 1                           # Start each bubble
        while i < n:
            if lyst[i] < lyst[i - 1]:   # Exchange if needed
                swap(lyst, i, i - 1)
            i += 1
        n -= 1
```

```
def bubbleSort2(lyst):
    n = len(lyst)
    while n > 1:
        swapped = False
        i = 1
        while i < n:
            if lyst[i] < lyst[i - 1]:  # Exchange if needed
                swap(lyst, i, i - 1)
                swapped = True
            i += 1
        if not swapped: return         # Return if no swaps
        n -= 1
```

11.4.3 Insertion Sort

- In the k-th pass, insert the k-th element into the already sorted sub-array of the first (k-1) elements to keep k first elements to be in order.

  - On the *i*th pass through the list, where *i* ranges from 1 to *n* – 1, the *i*th item should be inserted into its proper place among the first *i* items in the list.
  - After the *i*th pass, the first *i* items should be in sorted order.

```
def insertionSort(lyst):
    i = 1
    while i < len(lyst):
        itemToInsert = lyst[i]
        j = i - 1
        while j >= 0:
            if itemToInsert < lyst[j]:
                lyst[j + 1] = lyst[j]
                j -= 1
            else:
                break
        lyst[j + 1] = itemToInsert
        i += 1
```

UNSORTED LIST	AFTER 1st PASS	AFTER 2nd PASS	AFTER 3rd PASS	AFTER 4th PASS
2	2	1*	1	1
5 ←	5 (no insertion)	2	2	2
1	1←	5	4*	3*
4	4	4 ←	5	4
3	3	3	3 ←	5

[TABLE 11.5] A trace of the data during an insertion sort

Once again, analysis focuses on the nested loop. The outer loop executes $n - 1$ times. In the worst case, when all of the data are out of order, the inner loop iterates once on the first pass through the outer loop, twice on the second pass, and so on, for a total of $\frac{1}{2}\, n^2 - \frac{1}{2}\, n$ times. Thus, the worst-case behavior of insertion sort is $O(n^2)$.

The more items in the list that are in order, the better insertion sort gets until, in the best case of a sorted list, the sort's behavior is linear. In the average case, however, insertion sort is still quadratic.

11.4.4 Best-Case, Worst-Case, and Average-Case Performance Revisited

1. **Best case**—Under what circumstances does an algorithm do the least amount of work? What is the algorithm's complexity in this best case?
2. **Worst case**—Under what circumstances does an algorithm do the most amount of work? What is the algorithm's complexity in this worst case?
3. **Average case**—Under what circumstances does an algorithm do a typical amount of work? What is the algorithm's complexity in this typical case?

11.4.4 Best-Case, Worst-Case, and Average-Case Performance Revisited

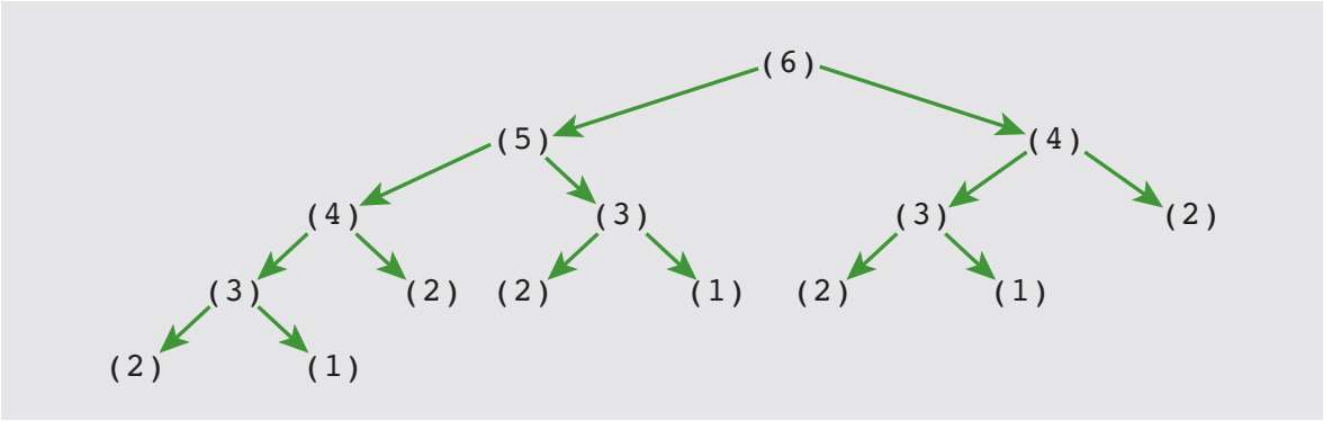
- Search for a minimum: best-case, worst-case, and average-case performances are O(n).
- Linear search: best-case performance is O(1) and its worst-case performance is O(n).
  - Average case: (1 + 2 + . . . + n) / n, or n / 2 comparisons; so O(n)
- Bubble sort's best-case performance is O(n); worst-case performance is clearly O(n^2).
  - Bubble sort's average-case performance is closer to O(n^2) than to O(n)

11.4 Exercises

1. Which configuration of data in a list causes the smallest number of exchanges in a selection sort? Which configuration of data causes the largest number of exchanges?
2. Explain the role that the number of data exchanges plays in the analysis of selection sort and bubble sort. What role, if any, does the size of the data objects play?
3. Explain why the modified bubble sort still exhibits $O(n^2)$ behavior on the average.
4. Explain why insertion sort works well on partially sorted lists.

11.5 An Exponential Algorithm: Recursive Fibonacci


```
def fib(n):
    """The recursive Fibonacci function."""
    if n < 3:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
```



[FIGURE 11.8] A call tree for **fib(6)**

11.6 Converting Fibonacci to a Linear Algorithm

```
def fib(n, counter):
    """Count the number of iterations in the Fibonacci
    function."""
    sum = 1
    first = 1
    second = 1
    count = 3
    while count <= n:
        counter.increment()
        sum = first + second
        first = second
        second = sum
        count += 1
    return sum

Problem Size     Iterations
           2              0
           4              2
           8              6
          16             14
          32             30
```

REVIEW QUESTIONS

1. Timing an algorithm with different problem sizes
   - a. can give you a general idea of the algorithm's run-time behavior
   - b. can give you an idea of the algorithm's run-time behavior on a particular hardware platform and a particular software platform
2. Counting instructions
   - a. provides the same data on different hardware and software platforms
   - b. can demonstrate the impracticality of exponential algorithms with large problem sizes

3 The expressions $O(n)$, $O(n^2)$, and $O(k^n)$ are, respectively,

- a exponential, linear, and quadratic
- b linear, quadratic, and exponential
- c logarithmic, linear, and quadratic

4 A binary search

- a assumes that the data are arranged in no particular order
- b assumes that the data are sorted

5 A selection sort makes at most

- a n^2 exchanges of data items
- b n exchanges of data items

6 The best-case behavior of insertion sort and modified bubble sort is

- a linear
- b quadratic
- c exponential

7 An example of an algorithm whose best-case, average-case, and worst-case behaviors are the same is

- a linear search
- b insertion sort
- c selection sort

8 Generally speaking, it is better

a to tweak an algorithm to shave a few seconds of running time

b to choose an algorithm with the lowest order of computational complexity

9 The recursive Fibonacci function makes approximately

- a n^2 recursive calls for problems of a large size n
- b 2^n recursive calls for problems of a large size n

10 Each level in a completely filled binary call tree has

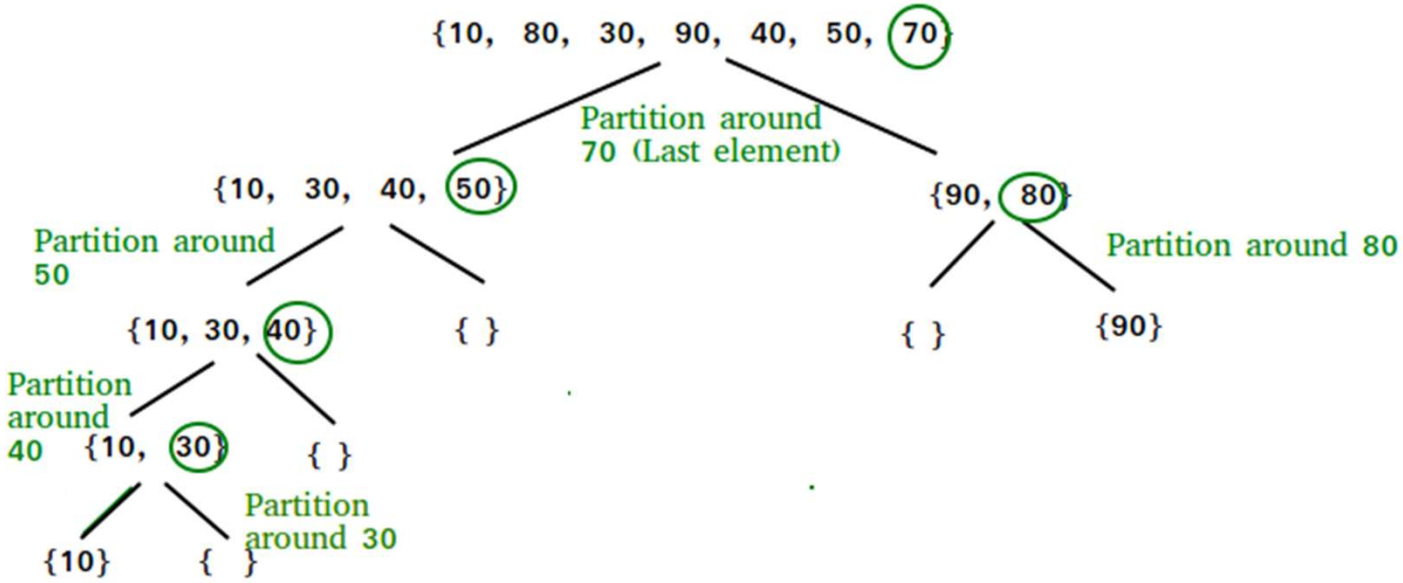
- a twice as many calls as the level above it
- b the same number of calls as the level above it

Faster Sorting

N	$n \log n$	n^2
512	4608	262,144
1024	10,240	1,048,576
2048	22,458	4,194,304
8192	106,496	67,108,864
16,384	229,376	268,435,456
32,768	491,520	1,073,741,824

Table 3-3 Comparing $n \log n$ and n^2

Quicksort


{10, 80, 30, 90, 40, 50, 70}
Partition around
70 (Last element)
{10, 30, 40, 50}
{90, 80}
Partition around
50
Partition around 80
{10, 30, 40}
{ }
{ }
{90}
Partition
around
40
{10, 30}
{ }
Partition
around 30
{10}
{ }

Quicksort

- QuickSort - Data Structure and Algorithm Tutorials - GeeksforGeeks
- 1) Selecting the item at the list's midpoint. This item is called the **pivot**.
- 2) Partition items in the list so that all items less than the pivot are moved to the left of the pivot, and the rest are moved to its right. The final position of the pivot itself varies, depending on the actual items involved.
- 3) Divide and conquer. Reapply the process recursively to the sublists formed by splitting the list at the pivot. One sublist consists of all items to the left of the pivot (now the smaller ones), and the other sublist has all items to the right (now the larger ones).
- 4) The process terminates each time it encounters a sublist with fewer than two items.

Partition

- Implement Various Types of Partitions in Quick Sort in Java - GeeksforGeeks
- 1. Swap the pivot with the last item in the sublist.
- 2. Establish a boundary between the items known to be less than the pivot and the rest of the items. Initially, this boundary is positioned immediately before the first item.
- 3. Starting with the first item in the sublist after the boundary, scan across the sublist. Every time you encounter an item less than the pivot, swap it with the first item after the boundary and advance the boundary.
- 4. Finish by swapping the pivot with the first item after the boundary.

Step	Action carried out	State of the list after the action									
	Let the sublist consist of the numbers shown with a pivot of 14.		12	19	17	18	14	11	15	13	16
1	Swap the pivot with the last item.		12	19	17	18	16	11	15	13	14
2	Establish the boundary before the first item.	:	12	19	17	18	16	11	15	13	14
3	Scan for the first item less than the pivot.	:	12	19	17	18	16	11	15	13	14
4	Swap this item with the first item after the boundary. In this example, the item gets swapped with itself.	:	12	19	17	18	16	11	15	13	14
5	Advance the boundary.	12	:	19	17	18	16	11	15	13	14
6	Scan for the next item less than the pivot.	12	:	19	17	18	16	11	15	13	14
7	Swap this item with the first item after the boundary.	12	:	11	17	18	16	19	15	13	14
8	Advance the boundary.	12	11	:	17	18	16	19	15	13	14
9	Scan for the next item less than the pivot.	12	11	:	17	18	16	19	15	13	14
10	Swap this item with the first item after the boundary.	12	11	:	13	18	16	19	15	17	14
11	Advance the boundary.	12	11	13	:	18	16	19	15	17	14
12	Scan for the next item less than the pivot; however, there is not one.	12	11	13	:	18	16	19	15	17	14
13	Interchange the pivot with the first item after the boundary. At this point, all items less than the pivot are to the pivot's left and the rest are to its right.	12	11	13	:	14	16	19	15	17	18

Figure 3-11 Partitioning a sublist

List Segment	Pivot Item
12 19 17 18 14 11 15 13 16	14
12 11 13	11
13 12	13
12	
16 19 15 17 18	15
19 18 17 16	18
16 17	16
17	
19	

Figure 3-12 Partitioning a sublist

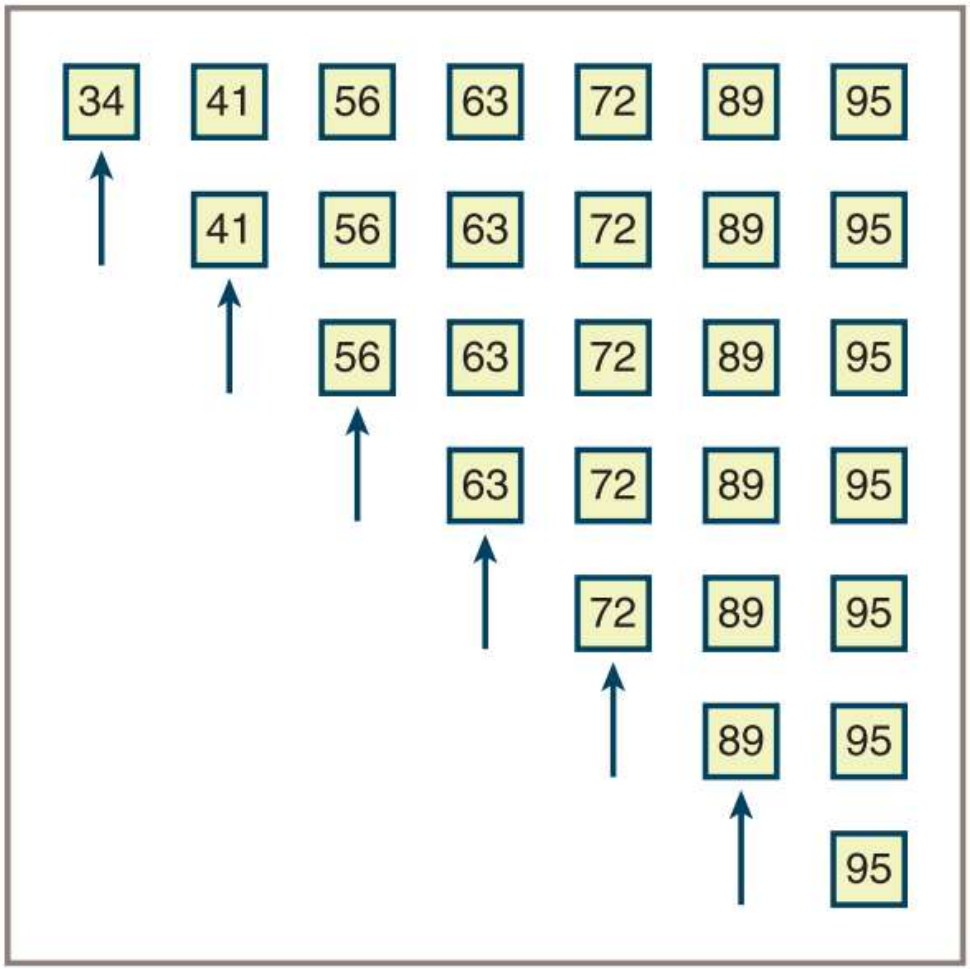


Figure 3-13 A worst-case scenario for quicksort (arrows indicate pivot elements)

Implementation of Quicksort


```
def quicksort(lyst):
    quicksortHelper(lyst, 0, len(lyst) - 1)

def quicksortHelper(lyst, left, right):
    if left < right:
        pivotLocation = partition(lyst, left, right)
        quicksortHelper(lyst, left, pivotLocation - 1)
        quicksortHelper(lyst, pivotLocation + 1, right)
```

```
def partition(lyst, left, right):
    # Find the pivot and exchange it with the last item
    middle = (left + right) // 2
    pivot = lyst[middle]
    lyst[middle] = lyst[right]
    lyst[right] = pivot
    # Set boundary point to first position
    boundary = left
    # Move items less than pivot to the left
    for index in range(left, right):
        if lyst[index] < pivot:
            swap(lyst, index, boundary)
            boundary += 1
    # Exchange the pivot item and the boundary item
    swap (lyst, right, boundary)
    return boundary
```

```
# Earlier definition of the swap function goes here

import random

def main(size = 20, sort = quicksort):
    lyst = []
    for count in range(size):
        lyst.append(random.randint(1, size + 1))
    print(lyst)
    sort(lyst)
    print(lyst)

if __name__ == "__main__":
    main()
```

Time and Space Complexity Analysis of Quick Sort - GeeksforGeeks

Time Complexity:

- **Best Case**: Ω (N log (N))
  The best-case scenario for quicksort occur when the pivot chosen at the each step divides the array into roughly equal halves.
  In this case, the algorithm will make balanced partitions, leading to efficient Sorting.
- **Average Case: θ (N log (N))**
  Quicksort's average-case performance is usually very good in practice, making it one of the fastest sorting Algorithm.
- **Worst Case: O(N2)**
  The worst-case Scenario for Quicksort occur when the pivot at each step consistently results in highly unbalanced partitions. When the array is already sorted and the pivot is always chosen as the smallest or largest element. To mitigate the worst-case Scenario, various techniques are used such as choosing a good pivot (e.g., median of three) and using Randomized algorithm (Randomized Quicksort) to shuffle the element before sorting.
- **Auxiliary Space:** O(1), if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Advantages of Quick Sort:

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.

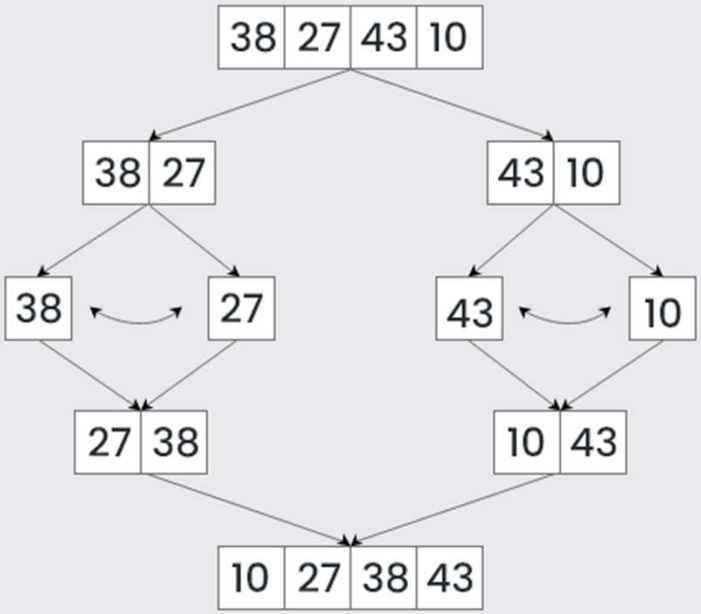
Disadvantages of Quick Sort:

- It has a worst-case time complexity of $O(N^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Merge Sort

Merge Sort

Algorithm



Merge Sort

- Merge Sort - Data Structure and Algorithms Tutorials - GeeksforGeeks
- Compute the middle position of a list and recursively sort its left and right sublists (divide and conquer).
- Merge the two sorted sublists back into a single sorted list.
- Stop the process when sublists can no longer be subdivided.

```
from arrays import Array

def mergeSort(lyst):
    # lyst          list being sorted
    # copyBuffer temporary space needed during merge
    copyBuffer = Array(len(lyst))
    mergeSortHelper(lyst, copyBuffer, 0, len(lyst) - 1)
```

```
def mergeSortHelper(lyst, copyBuffer, low, high):
    # lyst        list being sorted
    # copyBuffer  temp space needed during merge
    # low, high   bounds of sublist
    # middle      midpoint of sublist
    if low < high:
        middle = (low + high) // 2
        mergeSortHelper(lyst, copyBuffer, low, middle)
        mergeSortHelper(lyst, copyBuffer, middle + 1, high)
        merge(lyst, copyBuffer, low, middle, high)
```

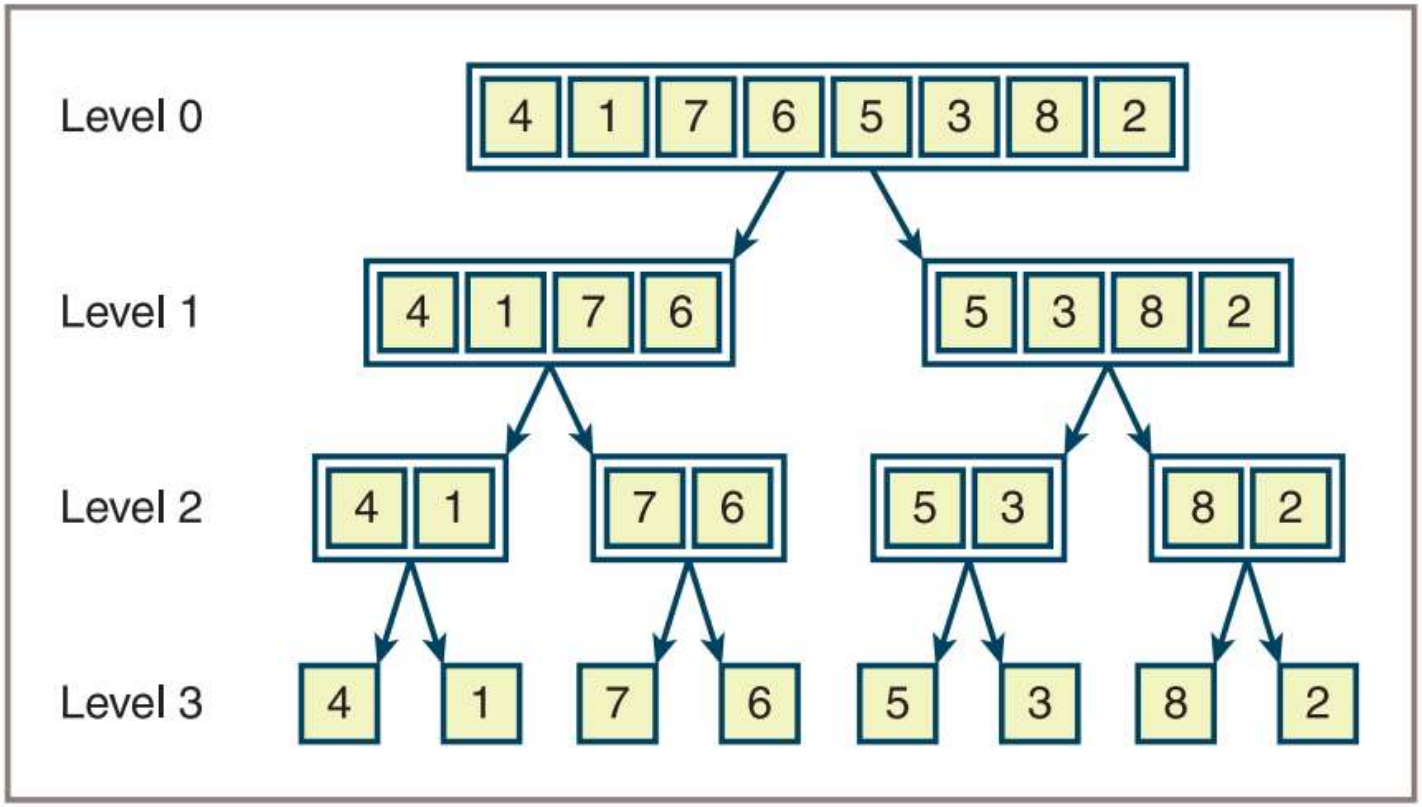


Figure 3-14 Sublists generated during calls of `mergeSortHelper`

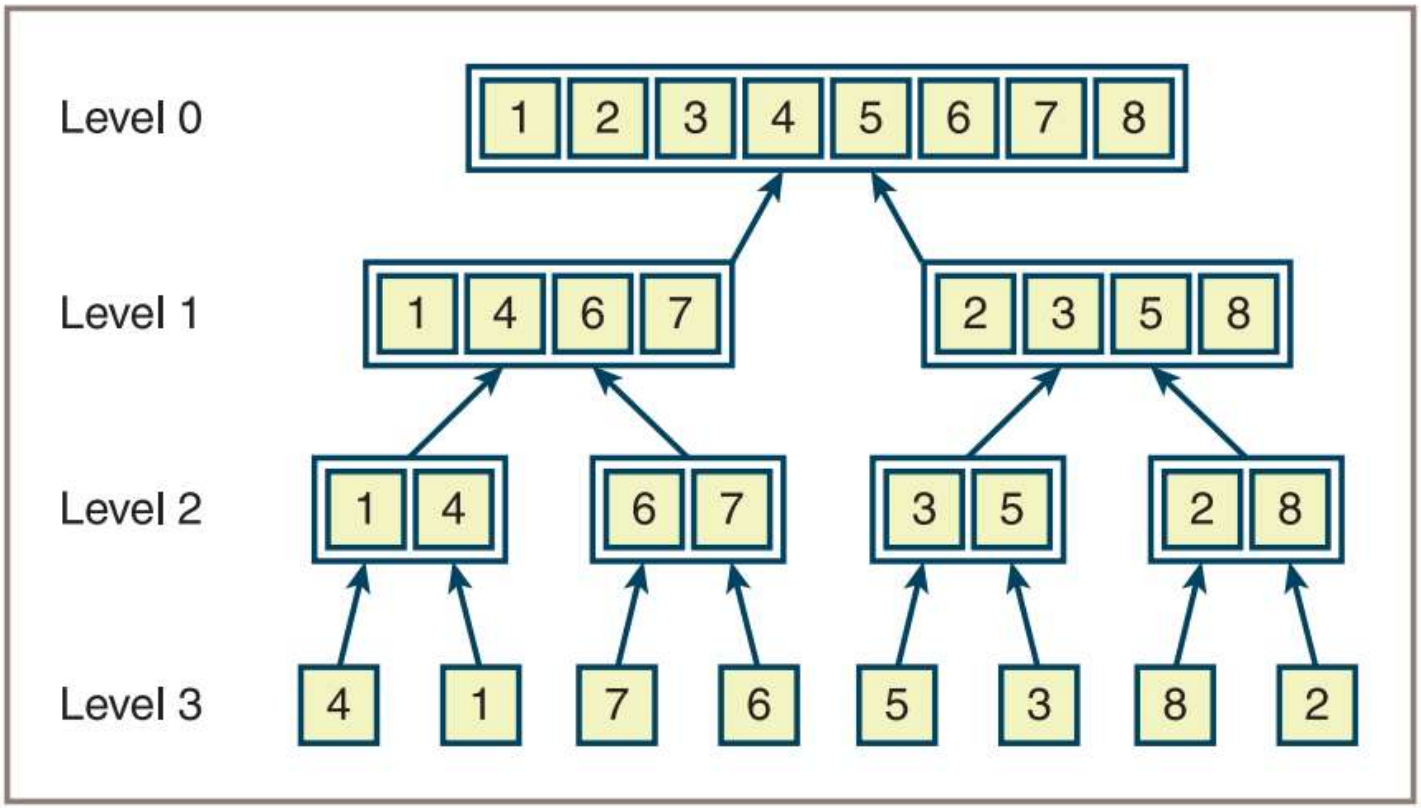


Figure 3-15 Merging the sublists generated during a merge sort

```
def merge(lyst, copyBuffer, low, middle, high):
    # lyst           list that is being sorted
    # copyBuffer     temp space needed during the merge proces
    # low            beginning of first sorted sublist
    # middle         end of first sorted sublist
    # middle + 1     beginning of second sorted sublist
    # high           end of second sorted sublist
    # Initialize i1 and i2 to the first items in each sublist
    i1 = low
    i2 = middle + 1
```

```
# Interleave items from the sublists into the
# copyBuffer in such a way that order is maintained.
for i in range(low, high + 1):
    if i1 > middle:
        copyBuffer[i] = lyst[i2]  # First sublist exhausted
        i2 += 1
    elif i2 > high:
        copyBuffer[i] = lyst[i1]  # Second sublist exhausted
        i1 += 1
    elif lyst[i1] < lyst[i2]:
        copyBuffer[i] = lyst[i1]  # Item in first sublist <
        i1 += 1
    else:
        copyBuffer[i] = lyst[i2]  # Item in second sublist <
        i2 += 1
for i in range (low, high + 1):   # Copy sorted items back to
    lyst[i] = copyBuffer[i]       # proper position in lyst
```

Complexity Analysis of Merge Sort

Time Complexity: O(N log(N)), Merge Sort is a recursive algorithm and time complexity can be expressed as following recurrence relation.

$$T(n) = 2T(n/2) + \theta(n)$$

The above recurrence can be solved either using the Recurrence Tree method or the Master method. It falls in case II of the Master Method and the solution of the recurrence is θ(Nlog(N)). The time complexity of Merge Sort isθ(Nlog(N)) in all 3 cases (worst, average, and best) as merge sort always divides the array into two halves and takes linear time to merge two halves.

Auxiliary Space: O(N), In merge sort all elements are copied into an auxiliary array. So N auxiliary space is required for merge sort.

Advantages of Merge Sort:

- **Stability**: Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance:** Merge sort has a worst-case time complexity of O(N logN), which means it performs well even on large datasets.
- **Parallelizable**: Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

Drawbacks of Merge Sort:

- **Space complexity:** Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place:** Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- **Not always optimal for small datasets:** For small datasets, Merge sort has a higher time complexity than some other sorting algorithms, such as insertion sort. This can result in slower performance for very small datasets.

Exercises

1. Describe the strategy of quicksort and explain why it can reduce the time complexity of sorting from $O(n^2)$ to $O(n \log n)$.
2. Why is quicksort not $O(n \log n)$ in all cases? Describe the worst-case situation for quicksort and give a list of 10 integers, 1–10, that would produce this behavior.
3. The **`partition`** operation in quicksort chooses the item at the midpoint as the pivot. Describe two other strategies for selecting a pivot value.
4. Sandra has a bright idea: when the length of a sublist in quicksort is less than a certain number—say, 30 elements—run an insertion sort to process that sublist. Explain why this is a bright idea.
5. Why is merge sort an $O(n \log n)$ algorithm in the worst case?

Practice

- Top MCQs on MergeSort Algorithm with Answers – GeeksforGeeks
- Practice | GeeksforGeeks | A computer science portal for geeks